

Introduction to Computer Organisation and Architecture UCCD1133



Chapter 4

Computer Architecture and Organization Fundamental

Disclaimer



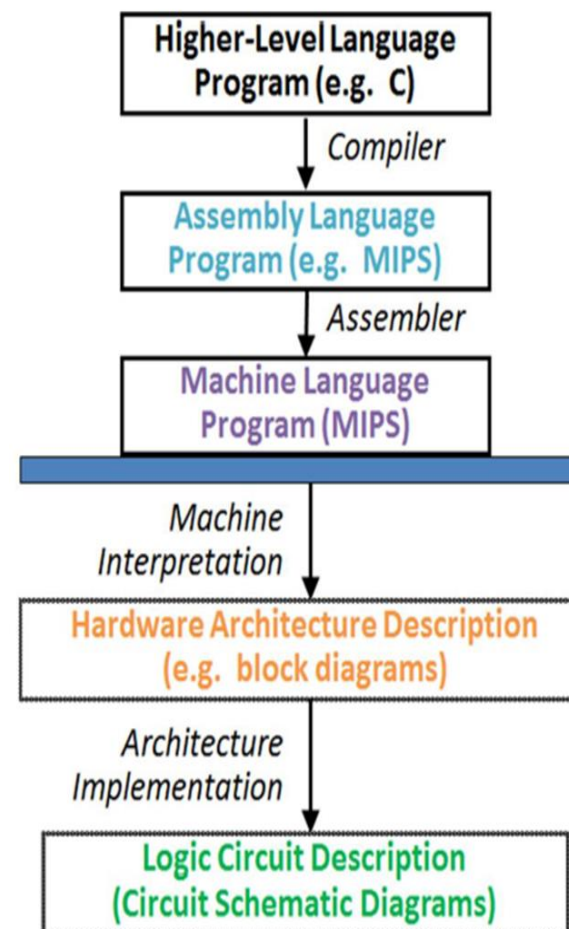
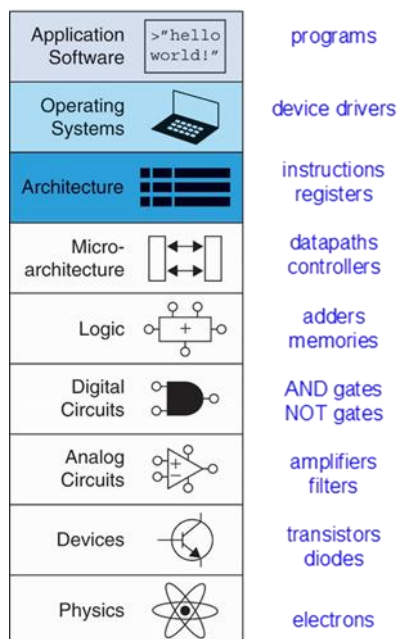
- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

Outline

- Overview
- Assembly Language
- Machine Language
- Programming
- Addressing Modes
- The Memory Map

Overview

- The architecture is the programmer's view of a computer.
- It is defined by the instruction set (language) and operand locations (registers and memory).



```

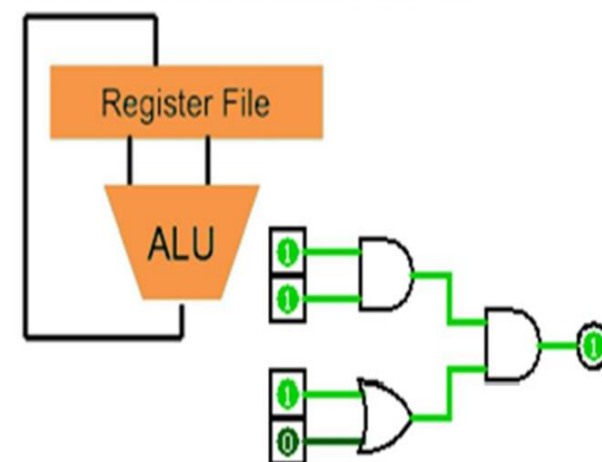
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
  
```

```

lw $t0, 0($2)
lw $t1, 4($2)
sw $t1, 0($2)
sw $t0, 4($2)
  
```

```

0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
  
```



Assembly language

- The words in a computer's language are called **instructions**.
- The computer's vocabulary is called the **instruction set**.
 - Assembly language: human-readable format of instructions
 - Machine language: computer-readable format (1's and 0's)
- **MIPS architecture**:
 - Developed by John Hennessy and his colleagues at Stanford and in the 1980's.
 - Used in many commercial systems, including Silicon Graphics, Nintendo, and Cisco
- Other instruction set architectures: Intel x86, ARM
- Computer instructions indicate both the **operation to perform** and the **operands to use**.
- The operands may come from memory, from registers, or from the instruction itself.

Assembly language

- Four design **principles of computer architecture**
 - Simplicity favors regularity
 - Make the common case fast
 - Smaller is faster
 - Good design demands compromise
- **MIPS assembly language**, examples of core instructions sets

Category	Instruction
Arithmetic	add (add), subtract (sub), add immediate (addi)
Data transfer	load word (lw), store word (sw), load byte (lb), store byte (sb), load upper immediate (lui)
Logical	and (and), or (or), nor (nor), and immediate (andi), or immediate (ori), shift left logical (sll), shift right logical (srl)
Conditional branch	branch on equal (beq), branch on not equal (bne), set on less than (slt)
Unconditional jump	Jump (j), jump register (jr), jump and link (jal)

Instructions: Arithmetic

- Addition

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code>add a, b, c</code>

- add, is called the mnemonic. Indicates what operation to perform.
- b and c, the source operands (on which the operation is performed on)
- a, the destination operand (the result is written to)

- Subtraction

High-Level Code	MIPS Assembly Code
<code>a = b - c;</code>	<code>sub a, b, c</code>

- Similar to addition, only the mnemonic change

1. Simplicity favors regularity

- MIPS has consistent instruction format (Eg add and sub)
- Same number of operands (two sources and one destination)
- Easier to encode and handle in hardware

Instructions: Arithmetic

- Multiple instruction
 - More complex code is handled by multiple MIPS instruction

High-Level Code	MIPS Assembly Code
<pre>a = b + c - d; // single-line comment /* multiple-line comment */</pre>	<pre>sub t, c, d #t = c - d add a, b, t #a = b + t</pre>

- assembly language, only single-line comments are used.
- The assembly language program above requires a temporary variable `t` to store the intermediate result.

2. Make the common case fast

- MIPS includes only simple, commonly used instructions
- Hardware to decode and execute instructions can be simple, small, and fast
- More complex instructions (that are less common) performed using multiple simple instructions
- MIPS is a **reduced instruction set computer (RISC)**, with a small number of simple instructions
- Other architectures, such as Intel's x86, are **complex instruction set computers (CISC)**

CISC vs RISC

	CISC	RISC
1	Variable instruction length	Fixed instruction length
2	Large number of addressing modes	Few addressing modes.
3	Support for small number of general purpose registers.	Support for large number of general purpose registers
4	Requires less number of instructions to represent an application code when compared to RISC.	Requires more number of instructions to represent an application code when compared to CISC although this is debatable
5	Requires complex Compiler.	Requires less complex Compiler
6	Since the application code compiled for CISC instruction set results in less number of instructions we need less memory to store the application binaries in a CISC machine.	Since the application code compiled for RISC instruction set results in more number of instructions (when compared to CISC) we need more memory to store the application binaries in a RISC machine
7	Less number of instructions need not necessarily mean that an application running on a CISC processor results in higher performance than the same running on a RISC processor.	More number of instructions need not necessarily mean that an application running on a RISC processor results in lower performance than the same running on a CISC processor
8	In addition to Load/Store there are other instructions which results in accessing memory.	Load/Store (Atomics included) are the only ones which can access memory
9	A typical CISC instruction (<i>Intel x 86 instruction</i>).	A typical RISC instruction (<i>SPARC instruction</i>)
10	Examples of CISC processors are Intel's 486, Pentium (all flavours), AMD's Krypton, Athlon etc.	Examples of RISC processors are SUN's UltraSparc, MIPS's MIPS32, MIPS64, ARM'S ARM11, Motorola's PowerPC etc

Operands: Registers

- An instruction operates on operands.

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code>add a, b, c</code>

- In code above the variables a, b, and c are all operands. But computers operate on 1's and 0's, not variable names.
- The instructions need a physical location from which to retrieve the binary data.
- Operands can be stored in physical location in computer
 - registers
 - memory
 - Constants (also known as immediate/stored in the instruction itself.)

- MIPS has 32 registers, 32-bit in size
- Registers are faster than memory
- MIPS is called “32-bit architecture” because it operates on 32-bit data

Name	Register Number	Usage
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	Function return values
\$a0-\$a3	4-7	Function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	Function return address

Registers

- Registers:
 - \$ before name
 - Example: \$0, “register zero”, “dollar zero”
- Registers used for specific purposes:
 - \$0 always holds the constant value 0.
 - the saved registers, \$s0-\$s7, used to hold variables
 - the temporary registers, \$t0 - \$t9, used to hold intermediate values during a larger computation
 - Discuss others later

3. Smaller is faster

- MIPS includes only a small number of registers
 - The fewer the registers, the faster they can be accessed.
 - A very large number of registers may increase the clock cycle time because it takes electronic signals longer when they must travel farther.

Instructions with Registers

- Revisit add instruction

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code>add a, b, c</code>

- Operands are replaced by registers

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code># \$s0 = a, \$s1 = b, \$s2 = c</code> <code>add \$s0, \$s1, \$s2 # a = b + c</code>

- The instruction adds the 32-bit values contained in \$s1 and \$s2 and writes the 32-bit result to \$s0.

- Register names beginning with \$s are called **saved registers**.

- Register names beginning with \$t are called **temporary registers**. They are used for storing temporary variables.

High-Level Code	MIPS Assembly Code
<code>a = b + c - d;</code>	<code># \$s0 = a, \$s1 = b, \$s2 = c, \$s3 = d</code> <code>sub \$t0, \$s2, \$s3 # t = c - d</code> <code>add \$s0, \$s1, \$t0 # a = b + t</code>

Instructions with Register

Example 1 : Translate the high-level code shown in Figure below into MIPS assembly code by assuming the variables are stored at saved register

$a = b - c + d;$

$e = a + x;$

Solution:

$\# \$s0 = a, \$s1 = b, \$s2 = c, \$s3 = d, \$s4 = e, \$s5 = x$

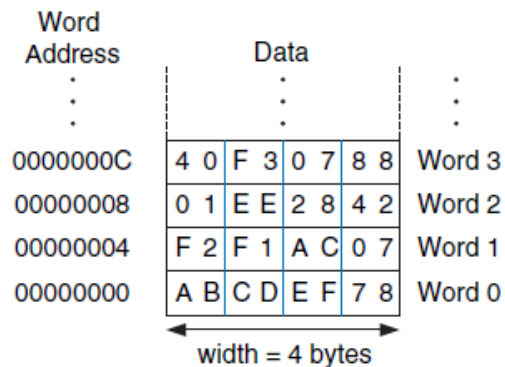
sub $\$t0, \$s1, \$s2$ $\#t0 = b - c$

add $\$s0, \$t0, \$s3$ $\#a = t0 + d$

add $\$s4, \$s0, \$s5$ $\#e = a + x$

Operands: Memory

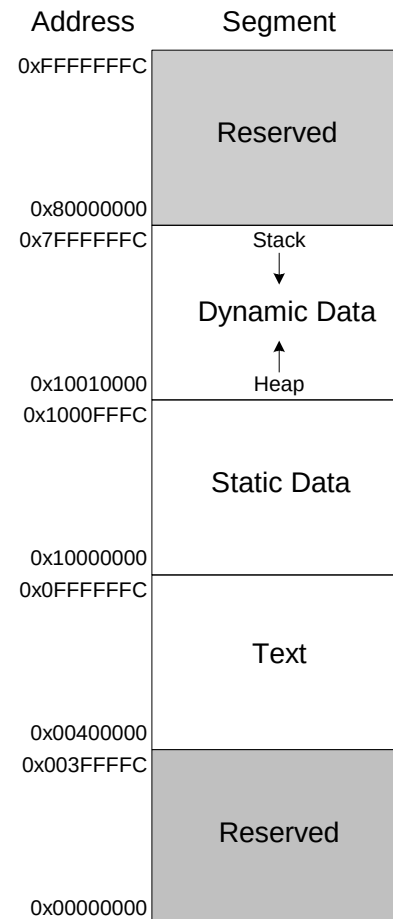
- Too much data to fit in only 32 registers
- Store more data in memory
- **Memory is large**, but **slow**
- Commonly used variables kept in registers
- Memories are organized as an array of data words.
- The MIPS architecture uses 32-bit memory addresses and 32-bit data words.



- MIPS uses a **byte-addressable memory**.
- Each data byte has unique address
- 32-bit word = 4 bytes, so word address increments by 4
- The address of a memory word must be multiplied by 4.
- For example:
 - the address of memory word 2 is $2 \times 4 = 8$
 - the address of memory word 10 is $10 \times 4 = 40$ (0x28)
- Both the 32-bit word address and the 32-bit data value are written in hexadecimal

Byte Addressable Memory

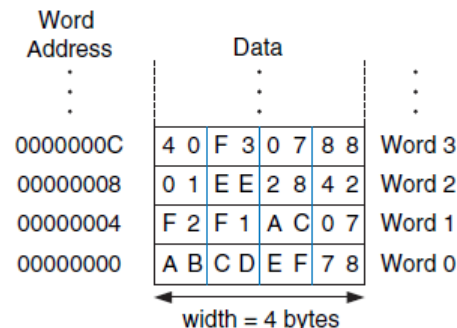
- A glance at the MIPS memory map



Instructions with Memory - Reading and Write the memory

- MIPS uses the **load word instruction, lw**, to read a data word from memory into a register.
- The lw instruction specifies the effective address in memory as the sum of a base address and an offset.
- The offset can be written in decimal or hexadecimal
- The base address (written in parentheses in the instruction) is a register
= \$0, which holds the value 0
- The offset is a constant (written before the parentheses)
= 4
- The lw instruction reads from effective address $(\$0 + 4) = 4$
- Word = $4/4 = 1$. Thus word 1
- After the load word instruction (lw) is executed, **\$s3 holds the value 0xF2F1AC07**

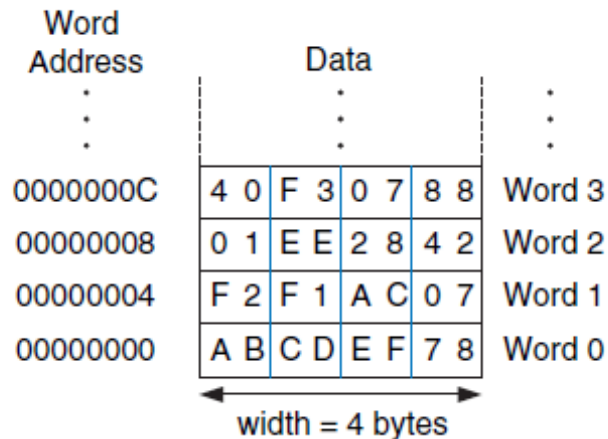
lw \$s3, 4(\$0)



Instructions with Memory - Reading and Write the memory

- MIPS uses the **store word instruction, sw**, to write a data word from a register into memory.

`sw $t7, 44($0)`



Address calculation:

- add base address (\$0) to the offset (5)
- Effective address = $(\$0 + 44) = 44$ or $0x2C$
- Word = $44 / 4 = 11$
- Result: write \$t7 value at word 11

Instructions with Memory - Reading and Write the memory

Example 2:

```
lw $s0, 0($0)
lw $s1, 8($0)
lw $s2, 0xC($0)
```

Word Address	Data								
⋮	⋮								⋮
0000000C	4	0	F	3	0	7	8	8	Word 3
00000008	0	1	E	E	2	8	4	2	Word 2
00000004	F	2	F	1	A	C	0	7	Word 1
00000000	A	B	C	D	E	F	7	8	Word 0

width = 4 bytes

Solution:

1. \$s0 hold the value of 0xABCDEF78
2. \$s1 hold the value of 0x01EE2842
3. \$s2 hold the value of 0x40F30788

Example 3:

```
sw $s3, 4($0)
sw $s4, 0x20($0)
sw $s5, 400($0)
```

Word Address	Data								
⋮	⋮								⋮
0000000C	4	0	F	3	0	7	8	8	Word 3
00000008	0	1	E	E	2	8	4	2	Word 2
00000004	F	2	F	1	A	C	0	7	Word 1
00000000	A	B	C	D	E	F	7	8	Word 0

width = 4 bytes

Solution:

1. Write \$s3 into address 0x04 (eg at word 1)
2. Write \$s4 into address 0x20 (eg at word 8)
3. Write \$s5 into address 0x190 (eg at word 100)

Instructions with Memory - Reading and Write the memory

- In the **MIPS architecture**, word addresses for lw and sw must be word aligned. **That is, the address must be divisible by 4.**
- Thus, the instruction lw \$s0, 7(\$0) is an illegal instruction. Some architectures, such as x86, allow non-word-aligned data reads and writes
- **MIPS requires strict alignment for simplicity.** Of course, byte addresses for load byte and store byte, lb and sb, need not be word aligned.

Big-Endian & Little-Endian Memory

- The MIPS architecture also provides these instructions:
 - lb** - load
 - sb** - store
- load and store single bytes in memory rather than words.
- They are similar to lw and sw
- Byte-addressable memories are organized in a big-endian or little-endian
- In both formats, the most significant byte (MSB) is on the left and the least significant byte (LSB) is on the right.
- In big-endian machines, bytes are numbered starting with 0 at the big (MSB) end
- In little-endian machines, bytes are numbered starting with 0 at the little (LSB) end.
- Word addresses are the same in both formats
- Only the addresses of bytes within a word differ.

Big-Endian

Byte Address			
⋮			
C	D	E	F
8	9	A	B
4	5	6	7
0	1	2	3
MSB		LSB	

Little-Endian

Byte Address			
⋮			
F	E	D	C
B	A	9	8
7	6	5	4
3	2	1	0
MSB		LSB	

Big-Endian & Little-Endian Memory

Example 4:

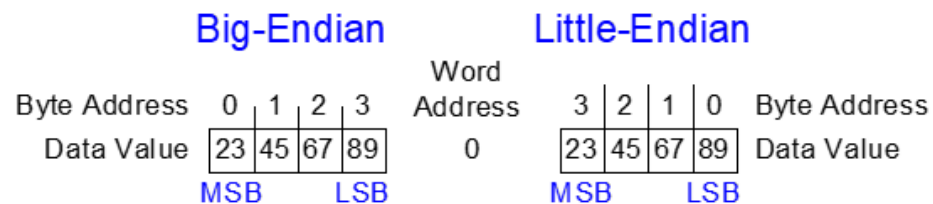
Suppose that \$s0 initially contains 0x23456789. After the following program is run on a big-endian system, what value does \$s0 contain? In a little-endian system?

```
sw $s0, 0($0)
```

```
lb $s0, 1($0)
```

Solution:

- Big-endian: 0x00000045
- Little-endian: 0x00000067



Operands: Constants/Immediates

- lw and sw also illustrate the use of constants in MIPS instructions.

```
lw $s3, 4($0)
```

- These constants are called **immediates**, because their values are immediately available from the instruction and **do not require a register or memory access**.
- Add immediate, addi, is another common MIPS instruction that uses an immediate operand.
- addi adds the immediate specified in the instruction to a value in a register as shown

High-Level Code	MIPS Assembly Code
a = a + 4; b = a - 12;	<pre># \$s0 = a, \$s1 = b addi \$s0, \$s0, 4 # a = a + 4 addi \$s1, \$s0, -12 # b = a - 12</pre>

- The immediate specified in an instruction is a 16-bit two's complement number in the range $[-32,768, 32,767]$.
- **Subtraction is equivalent to adding a negative number**, so, for simplicity, there is no **subi** instruction in the MIPS architecture.

Operands: Constants/Immediates

- Recall that the add and sub instructions use three register operands.
- But the lw, sw, and addi instructions use two register operands and a constant.
- lw and sw instructions violate design principle 1: simplicity favors regularity.
- However, this issue allows us to introduce the last design principle:

4. Good design demands good compromises

- MIPS Multiple instruction formats allow flexibility
 - add, sub: use 3 register operands
 - lw, sw: use 2 register operands and a constant

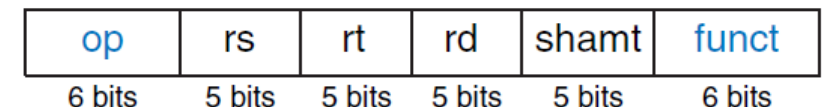
Machine language

- A program written in assembly language is translated from mnemonics to a representation using **only 1's and 0's called machine language**.
- MIPS uses 32-bit instructions
 - **Simplicity favors regularity: 32-bit data & instructions**
 - variable-length instructions would add too much complexity.
- MIPS compromises 3 instruction formats:
 - **R-Type**: register operands (operate on three registers.)
 - **I-Type**: immediate operand (operate on two registers and a 16-bit immediate)
 - **J-Type**: for jumping (operate on one 26-bit immediate.)

R-Type (Register-type)

- 3 register operands:
 - rs, rt : source registers
 - rd : destination register
- The 32-bit instruction has six fields: op, rs, rt, rd, shamt, and funct.
 - Op : the **operation code** or opcode (0 for R-type instructions)
 - Funct : the **function with opcode**, tells computer what operation to perform
 - Shamt : the **shift amount for shift instructions**, otherwise it's 0

R-type



Machine language (R-type)

- Assembly Code

```
add $s0, $s1, $s2
```

Note the order of registers in the assembly code:

```
sub $t0, $t3, $t5
```

```
add rd, rs, rt
```

- Field values

op	rs	rt	rd	shamt	funct
0	17	18	16	0	32
0	11	13	8	0	34
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- Machine Code

op	rs	rt	rd	shamt	funct	
000000	10001	10010	10000	00000	100000	(0x02328020)
000000	01011	01101	01000	00000	100010	(0x016D4022)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Example 5

Translate the following assembly language statement into machine language

```
add $t0, $s4, $s5
```

Solution

- Field values

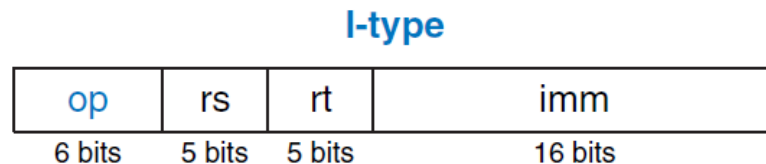
op	rs	rt	rd	shamt	funct
0	20	21	8	0	32
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- Machine Code

op	rs	rt	rd	shamt	funct	
000000	10100	10101	01000	00000	100000	(0x02954020)
0	2	9	5	4	0	2
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Machine language (I-type)

- I-type (Immediate type)
- I-type instructions use:
 - 2 register operands : rs, rt
 - 1 immediate operand
- The 32-bit instruction has six fields: op, rs, rt, and imm.
 - Op, rs and rt : similar to r-type instruction
 - Imm : 16-bit two's complement notation



- The operands are specified in the three fields rs, rt, and imm. Rs and imm are always used as source operands. rt is used as a destination for some instructions (such as addi and lw)

- Assembly Code and field values

Assembly Code	Field Values			
	op	rs	rt	imm
addi \$s0, \$s1, 5	8	17	16	5
addi \$t0, \$s3, -12	8	19	8	-12
lw \$t2, 32(\$0)	35	0	10	32
sw \$s1, 4(\$t1)	43	9	17	4
	6 bits	5 bits	5 bits	16 bits

Note the differing order of registers in assembly and machine codes:

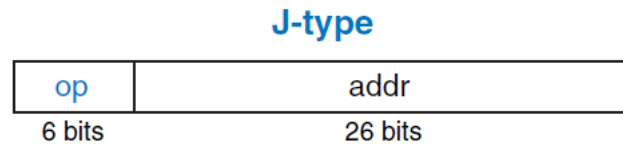
```
addi rt, rs, imm
lw   rt, imm(rs)
sw   rt, imm(rs)
```

- Machine Code

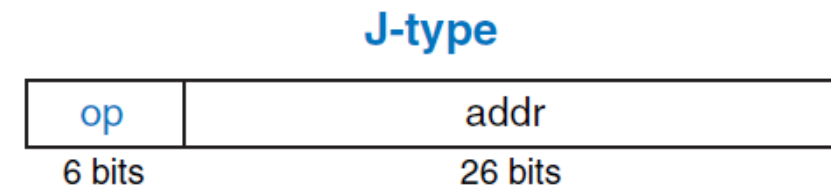
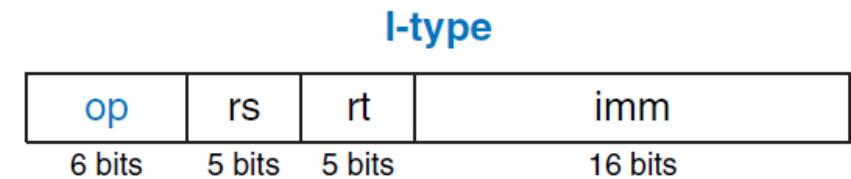
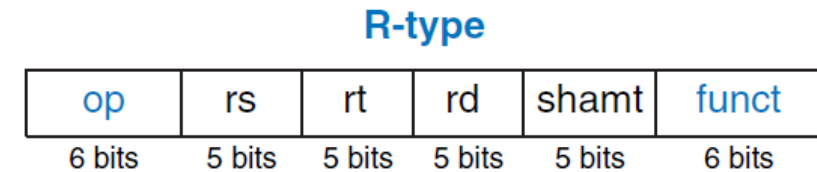
op	rs	rt	imm	
001000	10001	10000	0000 0000 0000 0101	(0x22300005)
001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
6 bits	5 bits	5 bits	16 bits	

Machine language (J-type)

- J-type (jump type)
- 26-bit address operand (addr)
- Used for jump instructions (j)



- Overall Instructions format in MIPS



Interpreting Machine Language Code

- Start with opcode: tells how to parse rest
- If opcode all 0's
 - R-type instruction
 - Function bits tell operation
- Otherwise
 - opcode tells operation

Name	Register Number	Usage
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	Function return values
\$a0-\$a3	4-7	Function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	Function return address

Machine Code

(0x2237FFF1)

op	rs	rt	imm
001000	10001	10111	1111 1111 1111 0001
2	2	3	7 F F F 1

Field Values

op	rs	rt	imm
8	17	23	-15

Assembly Code

addi \$s7, \$s1, -15

Machine Code

(0x02F34022)

op	rs	rt	rd	shamt	funct
000000	10111	10011	01000	00000	100010
0	2	F	3	4	0 2 2

Field Values

op	rs	rt	rd	shamt	funct
0	23	19	8	0	34

Assembly Code

sub \$t0, \$s7, \$s3

Power of the stored Program

- 32-bit instructions & data stored in memory
- This is called the **stored program** concept, and it is a key reason why computers are so powerful.
- To run a new program:
 - No rewiring required
 - Simply store new program in memory
- Program Execution:
 - Processor fetches (reads) instructions from memory in sequence
 - Processor performs the specified operation
- In MIPS programs, the instructions are normally stored **starting at address 0x00400000**.
- The address of the current instruction is kept in a 32-bit register called the **program counter (PC)**.

Assembly Code

Machine Code

lw	\$t2, 32(\$0)	0x8C0A0020
add	\$s0, \$s1, \$s2	0x02328020
addi	\$t0, \$s3, -12	0x2268FFF4
sub	\$t0, \$t3, \$t5	0x016D4022

Stored Program

Address	Instructions
⋮	⋮
0040000C	0 1 6 D 4 0 2 2
00400008	2 2 6 8 F F F 4
00400004	0 2 3 2 8 0 2 0
00400000	8 C 0 A 0 0 2 0
⋮	⋮

← **PC**
 Program Counter (PC): keeps track of current instruction

Main Memory

Exercise

```
lw $t0, 1200($t1)    # Temporary reg $t0 gets A[300]
add $t0, $s2, $t0     # $t0 gets h + A[300]
sw $t0, 1200($t1)    # Stores h + A[300] back into A[300]
```

What is the MIPS machine language code for these three instructions?

Exercise -answer

Answer:

lw \$t0, 1200(\$t1)

Step 1: Identify instruction format

Refer to MIPS Green card,
its I-Format instruction

Opcode: $23_{16} = 35_{10}$

Step 2: Map Operands to register number

Refer to MIPS Green card &
instruction syntax,

rs = \$t1 => \$9, rt = \$t0 => \$8

Step 3: Arrange opcode, reg number and imm value into instruction format & convert to hex code.

opcode								rs				rt				imm															
35								9				8				1200															
1	0	0	0	1	1	0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	1	0	0	1	0	1	1	0	0	0	0
8				D				2				8				0				4				B				0			

Ans: 0x 8D2804b0

Exercise- Answer

```
add $t0, $s2, $t0
```

opcode	rs	rt	rd	shamt	funct
0	18	8	8	0	32
0 0 0 0 0 0 1 0	0 0 1 0 0 1 0 0	0 1 0 0 0 0 0 0	0 1 0 0 0 0 0 0	0 0 0 0 0 0 0 0	1 0 0 0 0 0 0 0
0	2	4	8	4	0

Ans: 0x 02484020

```
sw $t0, 1200($t1)
```

opcode	rs	rt	imm
43	9	8	1200
1 0 1 0 1 1 0 1	0 0 1 0 0 1 0 1	0 1 0 0 0 0 0 0	0 0 0 0 0 0 1 0 0 1 0 1 1 0 0 0 0 0
A	D	2	8

Ans: 0x AD2804b0

Q & A

